

be rewritten in OOBASIC to avail the same functionality in OOo. This becomes a major issue in migrating to OOo. To overcome this, recent versions of OOo come with support of VBA interoperability. The Go-oo flavour of OOo is a better choice for this purpose.

Some background on Go-oo

Go-oo is Novell's effort, which is an addition to Sun's core. Some pending issues in the upstream have been fixed and it comes with some additional functionality. Here are a few advantages that the Go-oo people claim, when compared to other flavours of OOo.

- Quicker start-up
- VBA support and better Excel interoperability.
- A quick starter in the system tray.

For a complete list of the advantages, please visit go-oo.org/discover.

So, thinking of getting a copy of Go-oo? You will find that pre-built binaries for Linux, Mac and Windows are available from go-oo.mirrorbrain.org/stable.

At present, Go-oo support for VBA is to a much greater extent for Excel spreadsheets compared to only a little for Word documents. There's no VBA support for Power Point presentations, yet. You need to build Go-oo from sources to do some hacks or add VBA bits. The rest of this article explains how to build Go-oo and how to contribute by adding VBA properties and objects.

VBA support in OOo

The latest versions of upstream OOo and many versions of Go-oo come with VBA support. When an Excel document is opened in OOo, the VBA code used for macros is loaded as it is, with an additional line at the beginning—*Option VBASupport 1*. This tells the macro handler of OOo to process the code as per VBA specifications instead of default OOBASIC.

To enable/disable VBA Model, go to *Tools→Options→Load/Save→VBA Properties*. For each type of document, opt for load/save options. If the option 'Executable code' is not selected for Excel documents, the VBA code in Excel is loaded as comments when opened in OOo, just like in older versions that don't have VBA support.

OOo architecture

The OOo source is divided into modules. The advantage of this modular design is that each module can be rebuilt independently if you want to update the concerned libraries to reflect changes. For example, if a Calc related bug needs to be fixed, only the *sc* module should be rebuilt.

Around 200 modules exist in the current source code— at a location like *ooo-build/build/ooo3??-m??*. For example, the source tag *ooo310-m15* means OOo 3.1.0 version, with milestone 15.

Some of the important modules are:

- *sc* is for Calc
- *sd* is for Draw and Impress

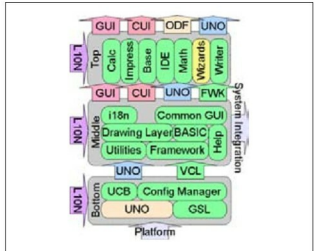


Figure 1: OOo architecture [Source : wiki.services.openoffice.org/wiki/Architecture]

- *sw* is for Writer
- *chart2* is for Calc charts
- *formula* is for math
- *oovbaapi* is for VBA support
- *sfx2* is a macro framework for Calc, Writer, Impress, etc.
- *vcl* is the UI Core

It is not advisable to change modules in the bottom or middle layers like *vcl*, *sfx*, etc. This is because changing these modules may lead to many side effects. Always try to fix issues by changing the top layer.

About ooo-build

Unlike the upstream OOo, one should obtain a copy of *ooo-build* first, as it is a wrapper for Sun's core. It contains many patches that are pending for upstream approval (though, of course, accepted by the Go-oo team) and additions to the core. On configuring *ooo-build*, a download script will retrieve the required sources. When you run *make* inside *ooo-build* the sources are extracted, patches are applied and then it prepares for an actual build. Thus, compared to the core, Go-oo enables faster integration of hacks by contributors and additional features like VBA.

Once a patch is approved by upstream at a later time, then only it's removed from *ooo-build* and merged into *core*.

Building Go-oo

The build takes around 8-10 hours on a machine with a typical configuration—and could take more or less time depending on hardware configuration. On a machine with an AMD Athlon 1700+ processor and 512 MB RAM, it takes around 10 hours. On a Core 2 Duo 2GHz processor with 3 GB RAM it takes around 4.5 hours.

Create two files – *t1* and *t2*, before and after a build in order to figure out the time taken.

```
touch t1 ; make ; touch t2
```

The difference between time stamps of the *t1* and *t2*